

Building the Ibex chip on IHP SG13G2 with Synopsys

The whole flow from the repository, step by step

Item	Value
Author	Cesar Valentin
Repository	https://github.com/XxMasterepicxX/ibex-sg13g2-chip
Chip	flash_chip: Ibex, two 4 KiB memories, a serial loader, a scan chain and 25 pads
Kit	IHP SG13G2, 130 nm
Tools	Fusion Compiler, StarRC, PrimeTime SI, Formality, PrimePower, TestMAX, VCS, SpyGlass, RedHawk-SC, and KLayout with IHP's rules

Contents

Before you start.....	3
Get the repository and set it up.....	3
What is in the repository.....	4
Stage 1: Make a run.....	4
Stage 2: Run it stage by stage.....	5
Stage 3: Synthesis and scan.....	6
Stage 4: Place and route.....	6
4.1 Floorplan and power grid.....	7
4.2 Placement and clock tree.....	8
4.3 Routing.....	9
4.4 Final checks and files.....	11
Stage 5: Finish the chip.....	12
Stage 6: Signoff.....	14
6.1 Wire delays, StarRC.....	15
6.2 Timing, PrimeTime.....	15
6.3 Logic, power, DRC and LVS.....	16
6.4 The checker.....	17
Stage 7: Margin rounds.....	17
7.1 Automatic rounds.....	18
7.2 Fixing it by hand.....	19
Stage 8: Proving it works.....	23
8.1 Programs on the gates.....	23
8.2 Scan test.....	24
8.3 Clock crossings.....	24
8.4 Voltage drop and electromigration.....	25
Stage 9: Review records and final signoff.....	26
Stage 10: The release package.....	27
Results of the fresh build.....	28
Labs: change one setting.....	29
Command summary.....	30

Before you start

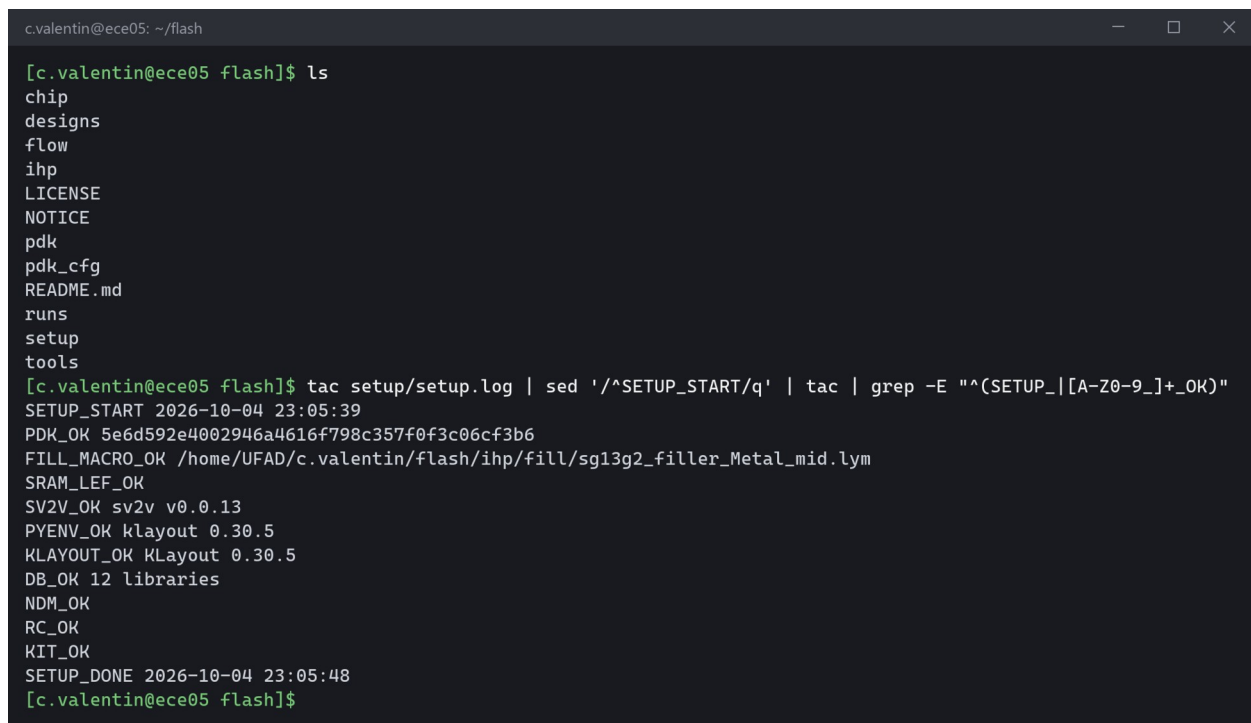
- Log in to a Linux server with the Synopsys and Ansys tools under /apps. Off campus, use the VPN.
- The scripts load the tools. To start a tool by hand, run `source /apps/settings` first.
- Fusion Compiler licenses are shared. When none is free, the build waits and tries again.
- Long steps run for hours. Start them under `nohup` so they keep running after you log out.

Get the repository and set it up

Clone the repository into `~/flash` and run `setup` once. Every script expects it there.

```
git clone https://github.com/XxMasterepicxX/ibex-sg13g2-chip.git ~/flash
bash ~/flash/setup/setup.sh
```

Setup takes about two hours. It downloads IHP's kit, builds KLayout, installs `sv2v` and the Python packages, and builds the Synopsys libraries and wire models from IHP's kit with your licenses. Those files are in Synopsys formats, so the repository does not hold them. Each step prints a line ending in `_OK`. If setup stops, the last line names the step and its log says why. Fix it and run `setup.sh` again. Steps that passed skip themselves.



```
c.valentin@ece05: ~/flash
[c.valentin@ece05 flash]$ ls
chip
designs
flow
ihp
LICENSE
NOTICE
pdk
pdk_cfg
README.md
runs
setup
tools
[c.valentin@ece05 flash]$ tac setup/setup.log | sed '/^SETUP_START/q' | tac | grep -E "^(SETUP_|[A-Z0-9_]+_OK)"
SETUP_START 2026-10-04 23:05:39
PDK_OK 5e6d592e4002946a4616f798c357f0f3c06cf3b6
FILL_MACRO_OK /home/UFAD/c.valentin/flash/ihp/fill/sg13g2_filler_Metal_mid.lym
SRAM_LEF_OK
SV2V_OK sv2v v0.0.13
PYENV_OK klayout 0.30.5
KLAYOUT_OK KLayout 0.30.5
DB_OK 12 libraries
NDM_OK
RC_OK
KIT_OK
SETUP_DONE 2026-10-04 23:05:48
[c.valentin@ece05 flash]$
```

Figure 1. The end of `setup.log` from a second run of `setup.sh`. Each step only checked its result.

Setup is done when `~/flash/setup/setup.log` ends with `SETUP_DONE`.

What is in the repository

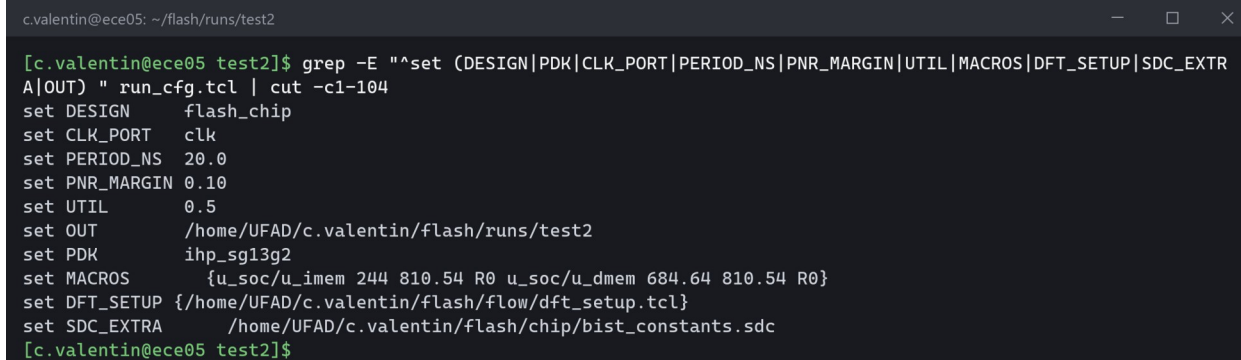
Folder	What it holds
setup/	the one-time setup
flow/	the Fusion Compiler flow <code>fc_flow.tcl</code> , <code>run_fc.sh</code> , <code>signoff.sh</code> , the checker <code>check.py</code> , chip finishing <code>finish.sh</code> and the fix loop <code>eco.sh</code>
pdk_cfg/ihp_sg13g2/	every fact about the IHP kit: layers, cells, corners, pads, bond pads and fill
ihp/	the scripts that build the Synopsys libraries, and the wire models
designs/flash_soc/	the RTL, the testbench and the test programs
chip/	the one-command build, margin rounds, proofs, review records and the release package
runs/<name>/	one folder per build, made by the scripts

Stage 1: Make a run

```
bash ~/flash/chip/new_run.sh my_chip
```

This makes `~/flash/runs/my_chip` with one settings file, `run_cfg.tcl`, copied from `chip/run_cfg.tcl` with the paths set to your home folder. Every later step reads it.

The screenshots come from fresh builds of this repository, named `test1`, `test2` and `walk`; each caption names its run when it matters. Where they show `test2` or `walk`, your folder is `my_chip`.



```
c.valentin@ece05: ~/flash/runs/test2
[c.valentin@ece05 test2]$ grep -E "^set (DESIGN|PDK|CLK_PORT|PERIOD_NS|PNR_MARGIN|UTIL|MACROS|DFT_SETUP|SDC_EXTR
A|OUT) " run_cfg.tcl | cut -c1-104
set DESIGN      flash_chip
set CLK_PORT    clk
set PERIOD_NS   20.0
set PNR_MARGIN  0.10
set UTIL        0.5
set OUT         /home/UFAD/c.valentin/flash/runs/test2
set PDK         ihp_sg13g2
set MACROS      {u_soc/u_imem 244 810.54 R0 u_soc/u_dmem 684.64 810.54 R0}
set DFT_SETUP   {/home/UFAD/c.valentin/flash/flow/dft_setup.tcl}
set SDC_EXTRA   /home/UFAD/c.valentin/flash/chip/bist_constants.sdc
[c.valentin@ece05 test2]$
```

Figure 2. The main settings of the run.

Setting	What it does
DESIGN	the top module, flash_chip: the core inside its pad ring
PERIOD_NS	the clock period signed off, 20 ns or 50 MHz
PNR_MARGIN	0.10 makes Fusion Compiler aim for a period 10% shorter, 18 ns, to leave room
CORE_SIZE	the core is 900 by 945 μm . With CORE_SIZE set, UTIL is not used
MAX_LAYER	the top routing layer, Metal5. Routing starts at Metal2, so the chip routes on 4 layers
MACROS	where the two memories go, in μm from the lower left corner
DFT_SETUP	how to build the scan chain
SDC_EXTRA	holds the memories' unused self-test clocks at 0 for timing
OUT	the run folder itself; run_fc.sh refuses to start if it is not

The chip must work at three corners. Each pairs a voltage and temperature with its library and wire model.

Corner	Voltage	Temperature	Checked mainly for	Wire model
slow	1.08 V	125 C	setup	worst
typ	1.2 V	25 C	power	typical
fast	1.32 V	-40 C	hold	best

Stage 2: Run it stage by stage

make_chip.sh runs stages 3 to 8 in order, and makes the run first if it does not exist. The first time, run it one stage at a time. STOP_AT stops it after the stage you name. Look at what that stage made, then run the same command with the next stage.

```
STOP_AT=synth bash ~/flash/chip/make_chip.sh my_chip
```

STOP_AT	Stops after
synth	stage 3, synthesis and scan, about 4 minutes
route	stage 4, place and route, about 12 minutes
finish	stage 5, chip finishing, about an hour
signoff	stage 6, the first signoff, about an hour and three quarters

After signoff, leave STOP_AT out. The build then runs stages 7 and 8 and stops at stage 9, when the review records are ready to sign. Steps that passed are skipped, so the same command picks up where the last one stopped. If a step fails, the log names it and says where to look. Each stage logs the tool command it runs, and the stages below show it too. Put MARGIN=hand in front to do stage 7 yourself.

Run the long stages under nohup and follow the log:

```
STOP_AT=signoff nohup bash ~/flash/chip/make_chip.sh my_chip > /dev/null 2>&1 &
tail -f ~/flash/runs/my_chip/make_chip.log
```

The commands for looking at results use RUN for the run folder:

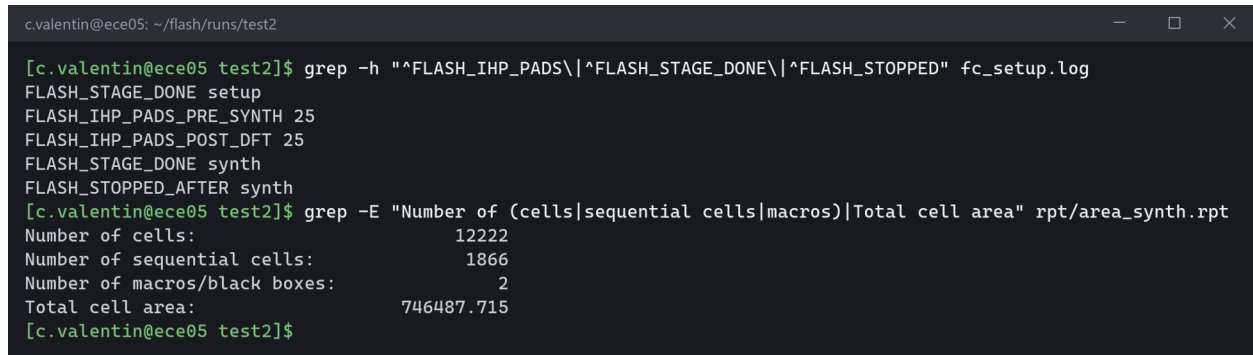
```
RUN=~/flash/runs/my_chip
```

Do not trust FC_EXIT=0 in a Fusion Compiler log. It prints even after an error. A session worked only if its log has FLASH_FLOW_DONE, or FLASH_STOPPED_AFTER when it was asked to stop.

Stage 3: Synthesis and scan

```
STOP_AT=synth bash ~/flash/chip/make_chip.sh my_chip
```

It runs STOP_AFTER=synth ~/flash/flow/run_fc.sh \$RUN. Fusion Compiler builds the RTL from IHP's standard cells and inserts the scan chain, which lets a tester load and read every flip-flop. The session stops after synthesis so you can check the result.

A terminal window showing the output of two grep commands. The first command searches for 'FLASH_IHP_PADS\|FLASH_STAGE_DONE\|FLASH_STOPPED' in the fc_setup.log file, showing a sequence of status updates: FLASH_STAGE_DONE setup, FLASH_IHP_PADS_PRE_SYNTH 25, FLASH_IHP_PADS_POST_DFT 25, FLASH_STAGE_DONE synth, and FLASH_STOPPED_AFTER synth. The second command searches for cell and area statistics in the rpt/area_synth.rpt file, displaying: Number of cells: 12222, Number of sequential cells: 1866, Number of macros/black boxes: 2, and Total cell area: 746487.715.

```
c.valentin@ece05: ~/flash/runs/test2
[c.valentin@ece05 test2]$ grep -h "^FLASH_IHP_PADS\|^FLASH_STAGE_DONE\|^FLASH_STOPPED" fc_setup.log
FLASH_STAGE_DONE setup
FLASH_IHP_PADS_PRE_SYNTH 25
FLASH_IHP_PADS_POST_DFT 25
FLASH_STAGE_DONE synth
FLASH_STOPPED_AFTER synth
[c.valentin@ece05 test2]$ grep -E "Number of (cells|sequential cells|macros)|Total cell area" rpt/area_synth.rpt
Number of cells: 12222
Number of sequential cells: 1866
Number of macros/black boxes: 2
Total cell area: 746487.715
[c.valentin@ece05 test2]$
```

Figure 3. The synthesis log and area report. All 25 pads survive scan insertion.

Synthesis is done when the log shows FLASH_IHP_PADS_POST_DFT 25. The netlist has 12,222 cells, 1,866 of them sequential, and the two memories.

Look at it yourself. The log tells the story of the session, and the area report counts the cells:

```
less $RUN/fc_setup.log
less $RUN/rpt/area_synth.rpt
```

Stage 4: Place and route

```
STOP_AT=route bash ~/flash/chip/make_chip.sh my_chip
```

It runs START_AT=floorplan ~/flash/flow/run_fc.sh \$RUN. This second session runs floorplan, pg, place, cts, route and final, and saves the design after each. START_AT restarts from a saved step.

4.1 Floorplan and power grid

The floorplan sets the die size, places the 25 pads in their ring and puts the two memories at the top of the core. The power grid brings VDD and VSS from the pad ring to every cell row.

```
c.valentin@ece05: ~/flash/runs/test2

[c.valentin@ece05 test2]$ grep -h "^FLASH_MACROS\|^FLASH_UNPLACED_PORTS\|^FLASH_IO_CORE_STRAPS\|^FLASH_PG_FLOATING" fc_floorplan.log
FLASH_MACROS u_soc/u_imem u_soc/u_dmem
FLASH_UNPLACED_PORTS 0
FLASH_IO_CORE_STRAPS VDD 36
FLASH_IO_CORE_STRAPS VSS 32
FLASH_PG_FLOATING_WIRES 0 0
[c.valentin@ece05 test2]$ grep -E "floating (wires|vias|hard macros|I/O pads)" rpt/pg_conn_pg.rpt | sort | uniq -c
      2 Number of floating hard macros: 0
      2 Number of floating I/O pads: 0
      2 Number of floating vias: 0
      2 Number of floating wires: 0
[c.valentin@ece05 test2]$
```

Figure 4. The floorplan and power grid reports. Both memories are placed, no port is unplaced and nothing floats.

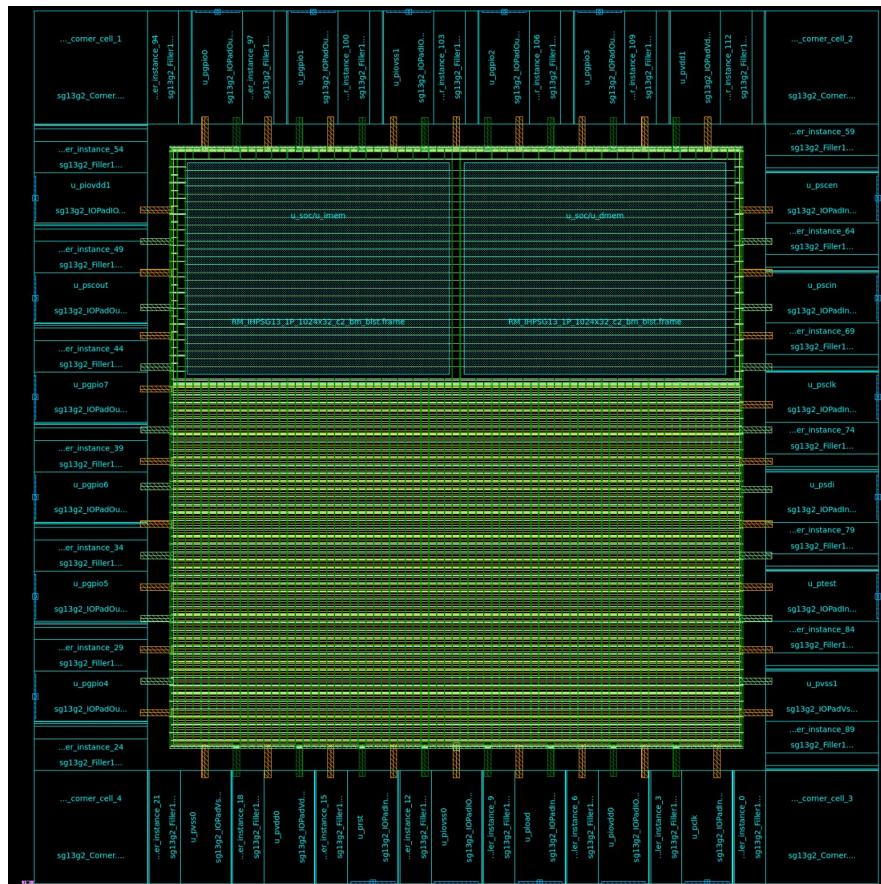


Figure 5. After the power grid: the pad ring, the two memories at the top, and the grid stripes below them.

The floorplan and power grid are done when FLASH_UNPLACED_PORTS is 0 and the power grid report shows no floating wires, vias, memories or pads.

4.2 Placement and clock tree

Placement puts every cell on a row and improves timing as it goes. `clock_opt` builds a conventional clock tree: it carries the clock to every flip-flop through buffers, so they all see it at nearly the same moment.

```
c.valentin@ece05: ~/flash/runs/test2

[c.valentin@ece05 test2]$ grep "succeeded\\|failed" rpt/legality_place.rpt | tail -1
check_legality succeeded.
[c.valentin@ece05 test2]$ grep -E "^(Clock /|Skew Group|clk |All Clocks)" rpt/clock_qor.rpt | head -4 | cut -c1-120
Clock /
rans DRC
Skew Group
Count
clk
0
clk
0
[c.valentin@ece05 test2]$ grep -h "^FLASH_CTS_ROUTE_DRC" fc_floorplan.log
FLASH_CTS_ROUTE_DRC 0
[c.valentin@ece05 test2]$
```

Clock /	Attrs	Sinks	Levels	Clock	Clock	Clock	Max	Global	T
rans DRC									
Skew Group				Repeater	Repeater	Stdcell	Latency	Skew	
Count									
clk	M,D	1865	10	461	3445.546	17876.391	1.122	0.561	
0									
clk	M,D	1865	10	461	3445.546	17876.391	1.122	0.561	
0									

Figure 6. A legal placement, a clock tree with 1,865 sinks and 10 buffer levels, and 0 wiring errors on the clock nets.

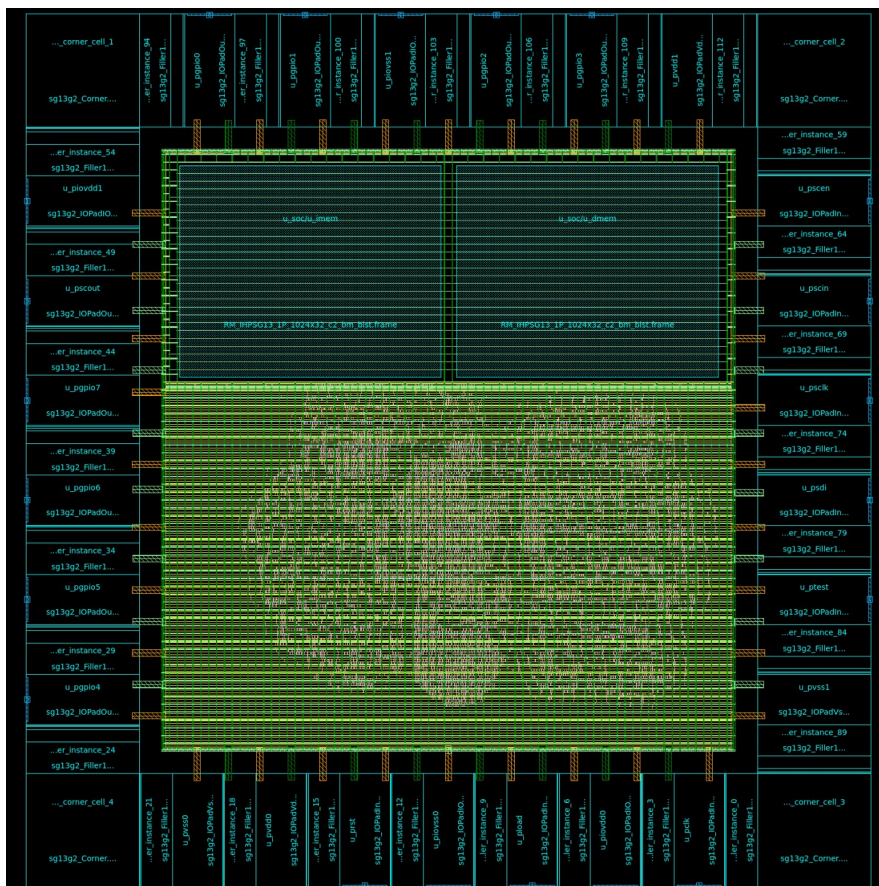


Figure 7. After placement. Every standard cell sits on a row below the memories.

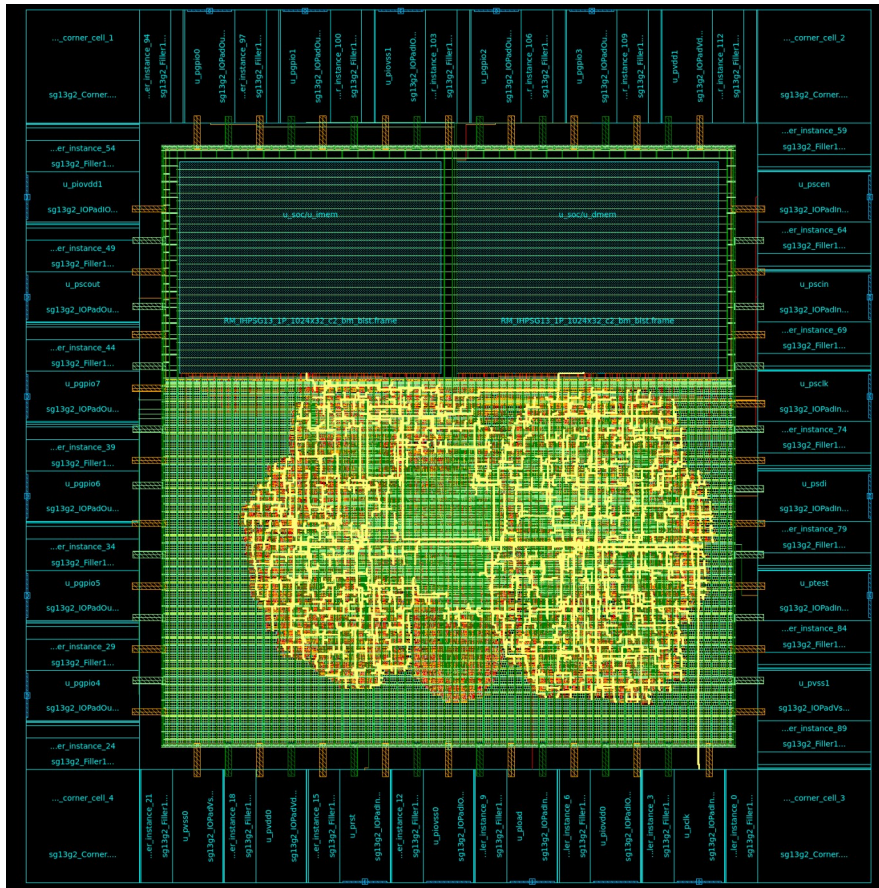


Figure 8. After the clock tree. The yellow clock nets spread from the clock pad to every flip-flop.

The clock wires go in first. The flow then adds decap cells, small capacitors that steady the supply, with their own Metal1, and reroutes the clock around them with route_eco. The log line FLASH_CTS_ROUTE_DRC must read 0.

4.3 Routing

Routing draws every connection in Metal2 to Metal5 and repairs its own rule errors.

```
c.valentin@ece05: ~/flash/runs/test2

[c.valentin@ece05 test2]$ grep -E "^Total number of (nets|open nets|DRCs|antenna)" rpt/check_routes.rpt | awk '!s[$0]++'
Total number of nets = 15909, of which 0 are not extracted
Total number of open nets = 0, of which 0 are frozen
Total number of DRCs = 0
Total number of antenna violations = 0
Total number of DRCs on Metal3 and below = 0
[c.valentin@ece05 test2]$ grep -E "^Total number of (short|open|floating)" rpt/check_lvs.rpt
Total number of short violations is 0.
Total number of open nets is 0.
Total number of floating route violations is 0.
[c.valentin@ece05 test2]$ grep -h "^FLASH_ROUTE_CLEANUP\|^FLASH_FLOW_DONE" fc_floorplan.log
FLASH_ROUTE_CLEANUP pass=1 drcs=0 signal_nets=0
FLASH_FLOW_DONE
[c.valentin@ece05 test2]$
```

Figure 9. The routing checks: no open nets, routing errors, antenna errors or shorts.

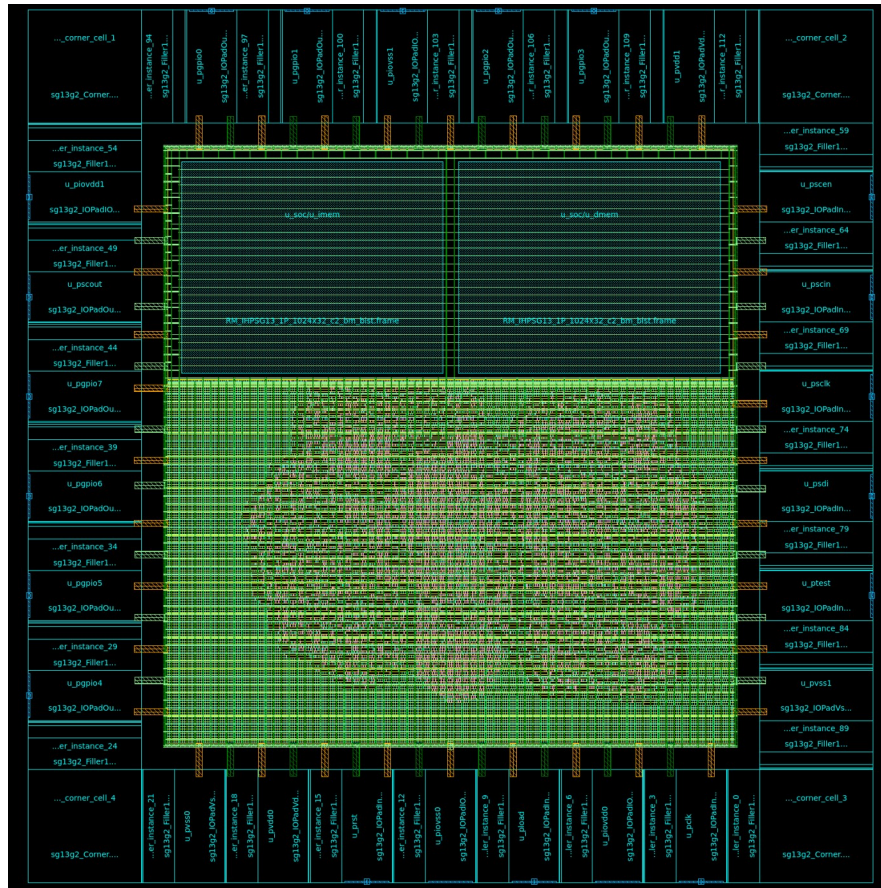


Figure 10. After routing. Signal wires on Metal2 to Metal5 cover the core.

Routing is done when check_routes reports 0 DRCs and 0 open nets, and the short check reports 0 shorts.

4.4 Final checks and files

```
c.valentin@ece05: ~/flash/runs/test2

[c.valentin@ece05 test2]$ grep -h "^FLASH_STAGE_DONE\|^FLASH_FLOW_DONE" fc_floorplan.log
FLASH_STAGE_DONE floorplan
FLASH_STAGE_DONE pg
FLASH_STAGE_DONE place
FLASH_STAGE_DONE cts
FLASH_STAGE_DONE route
FLASH_STAGE_DONE final
FLASH_FLOW_DONE
[c.valentin@ece05 test2]$ ls out
flash_chip.def
flash_chip.final.svf
flash_chip.floorplan.svf
flash_chip.gds
flash_chip.Internal_scan.spf
flash_chip.Internal_scan.synth.spf
flash_chip.pg.v
flash_chip.setup.svf
flash_chip.synth.v
flash_chip.v
[c.valentin@ece05 test2]$
```

Figure 11. Every step finished, and the files in out/: the layout, the netlists, the placed design and the inputs for Formality and the scan test.

Fusion Compiler judges timing against its 18 ns target. Stage 6 signs off the real 20 ns period with separate tools. Stage 4 is done when the log ends with FLASH_FLOW_DONE.

Look at it yourself. `open_fc.sh` opens any saved step read-only: floorplan, pg, place, cts, route or final. Ask Fusion Compiler for the timing summary and the worst path, then open the layout. The GUI needs an X display, such as MobaXterm or `ssh -X`.

```
bash ~/flash/chip/open_fc.sh $RUN final
report_qor -summary
report_timing
gui_start
```

```
c.valentin@ece05: ~/flash/runs/walk

[c.valentin@ece05 walk]$ printf "report_qor -summary\nexit\n" | bash ~/flash/chip/open_fc.sh . final | sed -n "/^Context/,/^Design *(Hold)/p"
Context                                     WNS      TNS      NVE
-----
func_slow      (Setup)      -0.11     -0.69      26
func_typ       (Setup)       6.09      0.00       0
Design         (Setup)      -0.11     -0.69      26

func_fast      (Hold)       0.00      0.00       0
func_typ       (Hold)       0.05      0.00       0
shift_fast     (Hold)       0.00      0.00       0
shift_slow     (Hold)       0.10      0.00       0
shift_typ      (Hold)       0.05      0.00       0
Design         (Hold)       0.00      0.00       0
[c.valentin@ece05 walk]$
```

Figure 12. `report_qor -summary` in `open_fc.sh` on the final step, per corner and mode.

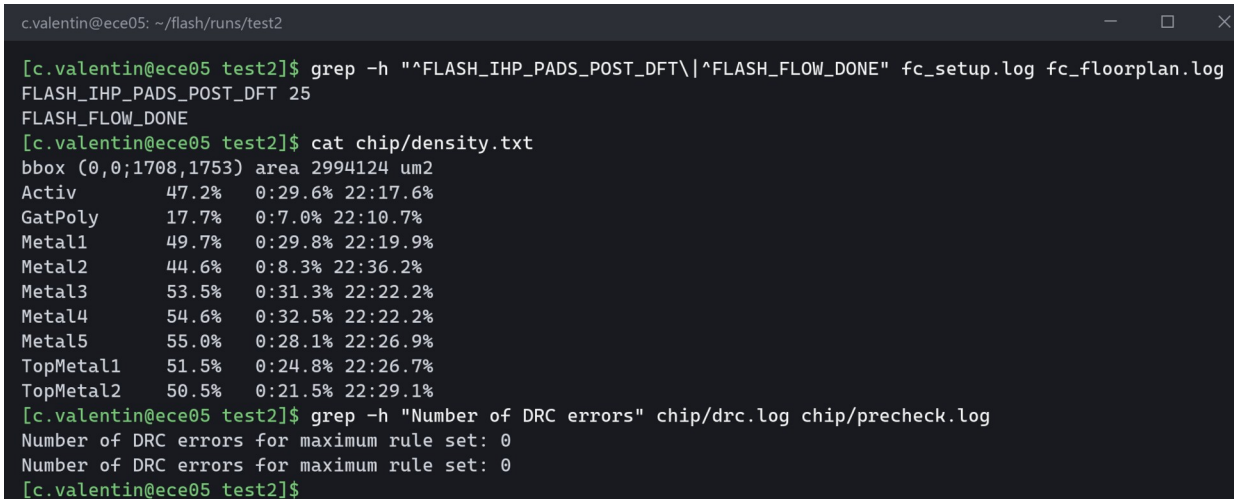
These numbers are against Fusion Compiler's own 18 ns target and 1 ns of clock uncertainty, so a small negative setup slack here is normal. Stage 6 signs off the real 20 ns period.

Stage 5: Finish the chip

```
STOP_AT=finish bash ~/flash/chip/make_chip.sh my_chip
```

It runs `METAL_FILL=mid bash ~/flash/flow/finish.sh $RUN 75 bondpads`. `finish.sh` puts a bond pad on every pad, a seal ring 75 μm out around the edge, and metal fill in every empty area, because the factory polishes each layer flat and needs even density. Then it runs whole-chip DRC and IHP's pre-check, the rule set IHP runs before it accepts a design. Both run at once. The step takes about an hour.

Before that, `make_chip.sh` records the signoff inputs that the chip DRC result is tied to, so run this stage through it rather than `finish.sh` alone.

A terminal window showing the execution of Stage 5. The user runs a command to grep for specific markers in log files, then cat's the density.txt file, and finally greps for DRC errors. The density.txt file shows various metal layers and their percentages. The DRC and precheck logs both show 0 errors.

```
c.valentin@ece05: ~/flash/runs/test2  
[c.valentin@ece05 test2]$ grep -h "^FLASH_IHP_PADS_POST_DFT\|^FLASH_FLOW_DONE" fc_setup.log fc_floorplan.log  
FLASH_IHP_PADS_POST_DFT 25  
FLASH_FLOW_DONE  
[c.valentin@ece05 test2]$ cat chip/density.txt  
bbox (0,0;1708,1753) area 2994124 um2  
Activ      47.2%    0:29.6% 22:17.6%  
GatPoly    17.7%    0:7.0%  22:10.7%  
Metal1     49.7%    0:29.8% 22:19.9%  
Metal2     44.6%    0:8.3%  22:36.2%  
Metal3     53.5%    0:31.3% 22:22.2%  
Metal4     54.6%    0:32.5% 22:22.2%  
Metal5     55.0%    0:28.1% 22:26.9%  
TopMetal1  51.5%    0:24.8% 22:26.7%  
TopMetal2  50.5%    0:21.5% 22:29.1%  
[c.valentin@ece05 test2]$ grep -h "Number of DRC errors" chip/drc.log chip/precheck.log  
Number of DRC errors for maximum rule set: 0  
Number of DRC errors for maximum rule set: 0  
[c.valentin@ece05 test2]$
```

Figure 13. The pad count, metal density and both DRC results. Every metal is inside IHP's limits and both DRC runs found 0 errors.

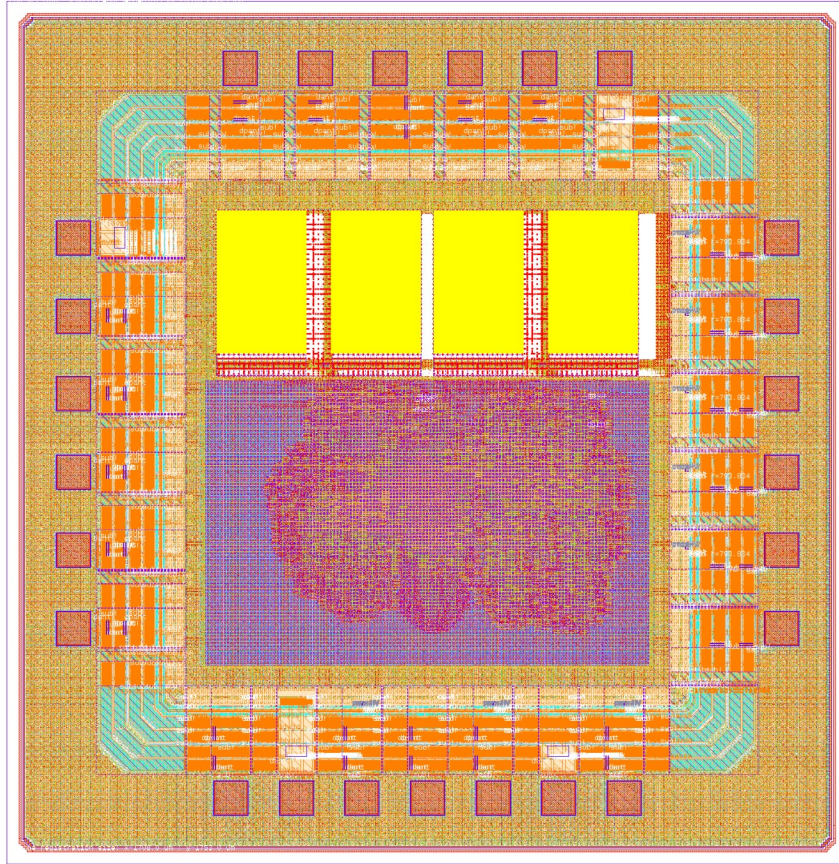


Figure 14. The finished chip in KLayout: the seal ring, 25 bond pads, the two memories in yellow, and fill everywhere else.

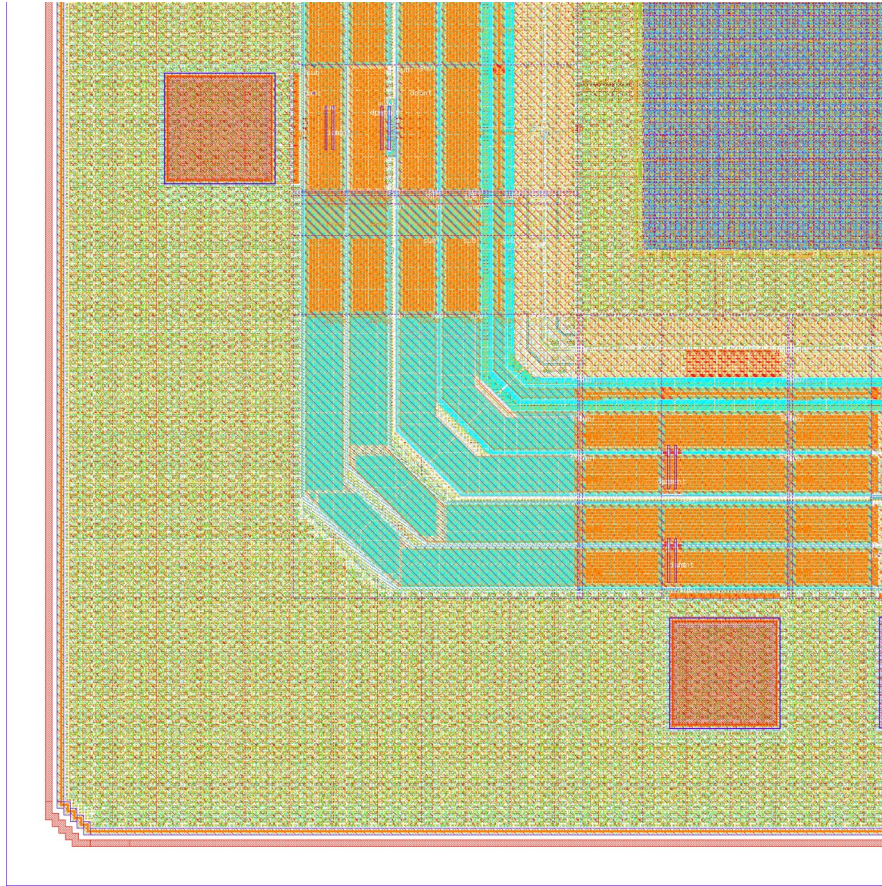


Figure 15. The lower left corner, 560 μm across. From the outside in: the seal ring, two bond pads, the pad ring's corner cell and the core edge. The fine pattern is metal fill.

IHP requires 35% to 60% metal on Metal1 to Metal5 and 25% to 70% on TopMetal1 and TopMetal2. Stage 5 is done when every layer in chip/density.txt is inside those limits and both DRC runs report 0 errors.

Look at it yourself in KLayout, with IHP's layer colors. Tools, Marker Browser loads the DRC results from \$RUN/chip/drc. Like the Fusion Compiler GUI, it needs an X display.

```
bash ~/flash/chip/open_klayout.sh $RUN
```

Stage 6: Signoff

```
STOP_AT=signoff nohup bash ~/flash/chip/make_chip.sh my_chip > /dev/null 2>&1 &
```

It runs ~/flash/flow/signoff.sh \$RUN. Signoff checks the design with tools that did not build it. StarRC extracts every wire with the typical, worst and best wire models. PrimeTime checks timing, crosstalk and electrical limits at three corners in normal and scan mode, six runs in all. Formality proves the netlist matches the RTL. PrimePower adds up the power. KLayout runs IHP's DRC and LVS on the block and LVS on the whole chip. Last, check.py writes one line per check to signoff/CHECK.txt.

The worst and best wire models come from the ranges in IHP's process specification, see `ihp/rc/DERIVATION.md`. IHP's open kit has DRC and LVS rules only for KLayout, Magic and Netgen, not for IC Validator, and IHP's tapeout check runs in KLayout, so DRC and LVS use KLayout. Fusion Compiler's own route and LVS checks in Stage 4 also pass. Signoff takes about an hour and three quarters.

6.1 Wire delays, StarRC

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ tail -qn1 starrc/starrc.log starrc_cmax/starrc.log starrc_cmin/starrc.log
STARRC_EXIT=0
STARRC_EXIT=0
STARRC_EXIT=0
[c.valentin@ece05 signoff]$ stat -c '%s bytes  %n' starrc*/flash_chip.spef
14079320 bytes  starrc_cmax/flash_chip.spef
14108263 bytes  starrc_cmin/flash_chip.spef
14072233 bytes  starrc/flash_chip.spef
[c.valentin@ece05 signoff]$
```

Figure 16. StarRC finished with all three wire models, one SPEF file each.

6.2 Timing, PrimeTime

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ for c in slow typ fast slow_shift typ_shift fast_shift; do printf "%-11s" $c; grep -
h "^FLASH_PT corner" pt_${c}/pt.log | head -1 | grep -o "setup_wns=[-0-9.]* hold_wns=[-0-9.]*"; done
slow      setup_wns=1.084774 hold_wns=0.149775
typ       setup_wns=7.745289 hold_wns=0.080328
fast      setup_wns=11.610882 hold_wns=0.038579
slow_shift setup_wns=1.107773 hold_wns=0.149813
typ_shift setup_wns=7.824506 hold_wns=0.080330
fast_shift setup_wns=11.642055 hold_wns=0.026272
[c.valentin@ece05 signoff]$ for c in slow typ fast; do grep -h "^FLASH_PT_NOISE" pt_${c}/pt.log; done
FLASH_PT_NOISE corner=slow worst_slack=0.03 violators=0
FLASH_PT_NOISE corner=typ worst_slack=0.04 violators=0
FLASH_PT_NOISE corner=fast worst_slack=0.03 violators=0
[c.valentin@ece05 signoff]$ grep -c VIOLATED pt_slow/violators.rpt pt_typ/violators.rpt pt_fast/violators.rpt
pt_slow/violators.rpt:3
pt_typ/violators.rpt:2
pt_fast/violators.rpt:2
[c.valentin@ece05 signoff]$
```

Figure 17. The PrimeTime runs. Setup, hold and noise pass everywhere. Each corner lists electrical violations.

Slow setup has +1.08 ns to spare. Fast hold has only +0.039 ns in normal mode and +0.026 ns in scan mode. That passes but leaves little room for differences between real chips. The electrical violations are two memory outputs driving slightly more load than allowed, and the flip-flop `u_soc/sc1k_s_reg[2]`. It uses only its inverted output, so its other output is under its smallest allowed load. Stage 7 fixes them.

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ awk '/Startpoint/{n++} n==1' pt_slow/setup.rpt | grep -E "Startpoint|Endpoint|Derati
ng|data arrival time|data required time|slack \" | sed 's/ */ /g' | awk '!s[$0]++' | cut -c1-104
Startpoint: u_soc/u_core/u_ibex_core/if_stage_i/instr_rdata_id_o_reg[23] (rising edge-triggered flip-fl
Endpoint: u_soc/u_core/u_ibex_core/if_stage_i/gen_prefetch_buffer.prefetch_buffer_i/stored_addr_q_reg[6
Max Data Paths Derating Factor : 1.05
Min Clock Paths Derating Factor : 0.95
Max Clock Paths Derating Factor : 1.05
data arrival time 19.33
data required time 20.42
data arrival time -19.33
slack (MET) 1.08
[c.valentin@ece05 signoff]$
```

Figure 18. The worst setup path at the slow corner. The data arrives at 19.33 ns and is needed by 20.42 ns. The derating factors are PrimeTime's 5% penalty on both sides.

6.3 Logic, power, DRC and LVS

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ grep -E "Verification (SUCCEEDED|FAILED)|Passing compare points|Failing compare poin
ts" formality/fm.log
Verification SUCCEEDED
2188 Passing compare points
[c.valentin@ece05 signoff]$ grep -E "Switching Power|Internal Power|Leakage Power|^Total Power" primepower/power
.rpt
Net Switching Power = 1.063e-03 (30.83%)
Cell Internal Power = 2.382e-03 (69.06%)
Cell Leakage Power = 3.738e-06 ( 0.11%)
Total Power = 3.449e-03 (100.00%)
[c.valentin@ece05 signoff]$
```

Figure 19. Formality: every compare point matches the RTL. PrimePower: the chip's total power.

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ grep -h "Number of DRC errors" drc.log
Number of DRC errors for maximum rule set: 0
[c.valentin@ece05 signoff]$ grep -h "Congratulations" lvs.log lvs_chip/lvs.log | sed 's/^.*INFO : //'
Congratulations! Netlists match.
Congratulations! Netlists match.
[c.valentin@ece05 signoff]$ grep -E "^(drc|lvs|lvs_completion|chip_lvs_completion) " CHECK.txt | cut -c1-104
drc PASS klayout violations=0 recommended_only=0 chip_level=GFi
lvs PASS netlists match
lvs_completion PASS latest match verdict and successful exit required
chip_lvs_completion PASS latest match verdict and successful exit required
[c.valentin@ece05 signoff]$
```

Figure 20. KLayout with IHP's rules. Block DRC finds 0 errors, and LVS matches for the block and the whole chip.

6.4 The checker

```
c.valentin@ece05: ~/flash/runs/test2/signoff
[c.valentin@ece05 signoff]$ grep -v " PASS " CHECK.txt | cut -c1-104
reports_atpg FAIL missing=signoff/atpg/evidence.json,signoff/atpg/summary
provenance_atpg MISSING evidence missing; no runtime fingerprint
electrical_slow FAIL unwaived=2 waived=0 first=u_soc/u_imem/A_DOUT[15]
timing_constraints_slow FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_slow FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_slow FAIL missing reviewed timing coverage contract
electrical_slow_shift FAIL unwaived=2 waived=0 first=u_soc/u_imem/A_DOUT[15]
timing_constraints_slow_shift FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_slow_shift FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_slow_shift FAIL missing reviewed timing coverage contract
electrical_typ FAIL unwaived=1 waived=0 first=u_soc/u_imem/A_DOUT[15]
timing_constraints_typ FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_typ FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_typ FAIL missing reviewed timing coverage contract
electrical_typ_shift FAIL unwaived=1 waived=0 first=u_soc/u_imem/A_DOUT[15]
timing_constraints_typ_shift FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_typ_shift FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_typ_shift FAIL missing reviewed timing coverage contract
electrical_fast FAIL unwaived=1 waived=0 first=u_soc/u_imem/A_DOUT[15]
timing_constraints_fast FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_fast FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_fast FAIL missing reviewed timing coverage contract
electrical_fast_shift FAIL unwaived=1 waived=0 first=u_soc/u_imem/A_DOUT[15]
timing_constraints_fast_shift FAIL violations=1 first=u_soc/sclk_s_reg[2]/Q 0.00
clock_coverage_fast_shift FAIL Warning: There are 2 register clock pins with no clock
pt_report_analysis_coverage_fast_shift FAIL missing reviewed timing coverage contract
formality_coverage FAIL passing=2188 excluded=1 clock_gate_latches=0; requires
atpg_coverage FAIL [Errno 2] No such file or directory: '/ecel/UFAD/c.vale
SIGNOFF NOT CLEAN: 111 of 139 checks pass
[c.valentin@ece05 signoff]$
```

Figure 21. The checks still failing after the first signoff. 111 of 139 pass.

The failing rows are the scan test, which is stage 8, the unsigned review records, stage 9, and the electrical and constraint rows from 6.2, which stage 7 fixes. Setup, hold, noise, Formality, DRC and both LVS runs already pass.

Look at it yourself. Every check is one line of CHECK.txt, and each PrimeTime run keeps its reports in its own folder:

```
less $RUN/signoff/CHECK.txt
less $RUN/signoff/pt_slow/setup.rpt
less $RUN/signoff/pt_fast/hold.rpt
```

Stage 7: Margin rounds

The hold floor is +0.09 ns of hold slack at the fast corner. The first signoff usually leaves fast hold under it, along with the 6.2 violations or a few noise victims. Any fix can break another corner. PrimeTime fixes hold at the fast corner without seeing the slow one. So every list of changes is tested at all three corners in both modes before Fusion Compiler applies it.

By default make_chip.sh runs the rounds for you. With MARGIN=hand it stops after the first signoff and you do the same work yourself.

7.1 Automatic rounds

```
bash ~/flash/chip/margin_fix.sh $RUN
```

1. PrimeTime proposes noise, load and hold fixes and changes nothing.
2. The list is tested at every corner and mode. A changed cell that leaves a path under +0.6 ns at slow or +0.2 ns at typ and fast is dropped, and the shorter list is tested again.
3. Targeted fixes cover the rest. A noise victim's driver gets a buffer after it. A driver over its limit gets one size larger. A hold endpoint gets a delay cell in front of it.
4. Fusion Compiler applies the list only if slow setup keeps +0.6 ns and fast hold reaches the hold floor. The chip is finished and signed off again, and the round is undone if any check that passed before fails.

make_chip.sh runs up to three rounds, each kept in \$RUN/margin/roundN.

```
c.valentin@ece05: ~/flash/runs/test1/margin/round1

[c.valentin@ece05 round1]$ grep -E "cand0 changes|FILTER|targeted pass|apply" ../../make_chip.log | sed -E 's/^0[0-9-]+ [0-9-:]+ //'
cand0 changes: 298
FILTER bad_cells 2 dropped 2 kept 296
targeted pass 1: 14 changes
targeted pass 2: 10 changes
apply 320 changes
[c.valentin@ece05 round1]$ sed -E 's/FLASH_EVAL //; s/ changed=[0-9]+//; s/([0-9]\.[0-9]{3})[0-9]+\1/g' cand0.e
val
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.142
corner=slow mode=func bad=2 drc_viol=14 noise_viol=0 setup_wns=0.802 hold_wns=0.389
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.240
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.142
corner=slow mode=shift bad=2 drc_viol=14 noise_viol=0 setup_wns=0.802 hold_wns=0.389
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.240
[c.valentin@ece05 round1]$
```

Figure 22. Round 1 on test1. Of 298 proposed changes, 2 left a slow path under +0.6 ns, and the test dropped both.

```
c.valentin@ece05: ~/flash/runs/test1/margin/round1

[c.valentin@ece05 round1]$ head -3 targeted1.tcl | cut -c1-80
insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 -new_net_n
insert_buffer [get_pins {u_soc/u_dmem/A_BM[8]}] sg13g2_dlygate4sd1_1 -new_net_na
insert_buffer [get_pins {u_soc/u_dmem/A_BM[9]}] sg13g2_dlygate4sd1_1 -new_net_na
[c.valentin@ece05 round1]$ grep -h "^size_cell" targeted1.tcl | head -2
size_cell {u_soc/u_core/u_ibex_core/A4312} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4318} {sg13g2_nand2_2}
[c.valentin@ece05 round1]$ sed -E 's/FLASH_EVAL //; s/ changed=[0-9]+//; s/([0-9]\.[0-9]{3})[0-9]+\1/g' cand3.e
val
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.113
corner=slow mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=0.802 hold_wns=0.339
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.202
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.468 hold_wns=0.113
corner=slow mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=0.802 hold_wns=0.339
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.579 hold_wns=0.202
[c.valentin@ece05 round1]$ cat gate.txt
MARGIN_GATE PASS
[c.valentin@ece05 round1]$
```

Figure 23. The targeted fixes and the final test of all 320 changes. Fast hold is +0.113 ns, slow setup keeps +0.802 ns, and no corner has a bad cell, electrical or noise violation.

```
c.valentin@ece05: ~/flash/runs/test1/margin/round1

[c.valentin@ece05 round1]$ grep -E "FLASH_ECO_(APPLIED|LEGALITY|KEPT|REJECTED|FAILED)" apply.log
FLASH_ECO_LEGALITY_PASS
FLASH_ECO_APPLIED 355
FLASH_ECO_KEPT 123 checks pass against 117 before
[c.valentin@ece05 round1]$
```

Figure 24. The verdict. The placement stayed legal and 123 checks pass against 117 before, so the round is kept.

7.2 Fixing it by hand

```
MARGIN=hand bash ~/flash/chip/make_chip.sh my_chip
```

The build stops after the first signoff and prints what needs fixing. The names below come from test2. Read your own reports at each step.

```
c.valentin@ece05: ~/flash/runs/test2

[c.valentin@ece05 test2]$ grep -E "^(electrical|timing_constraints)_(slow|fast) " signoff/CHECK.txt | cut -c1-96
electrical_slow          FAIL      unwaived=2 waived=0 first=u_soc/u_imem/A_DOUT[1
timing_constraints_slow   FAIL      violations=1 first=u_soc/sclk_s_reg[2]/Q
electrical_fast          FAIL      unwaived=1 waived=0 first=u_soc/u_imem/A_DOUT[1
timing_constraints_fast    FAIL      violations=1 first=u_soc/sclk_s_reg[2]/Q
[c.valentin@ece05 test2]$ grep -h "FLASH_PT corner=fast" signoff/pt_fast/pt.log signoff/pt_fast_shift/pt.log |
cut -c1-80
FLASH_PT corner=fast period=20.0 setup_wns=11.610882 hold_wns=0.038579 setup_wns
FLASH_PT corner=fast period=20.0 setup_wns=11.642055 hold_wns=0.026272 setup_wns
[c.valentin@ece05 test2]$ grep VIOLATED signoff/pt_slow/violators.rpt | cut -c1-80
u_soc/u_imem/A_DOUT[15]    0.06      0.07      0.00 (VIOLATED: i
u_soc/u_dmem/A_DOUT[25]    0.06      0.06      0.00 (VIOLATED: i
u_soc/sclk_s_reg[2]/Q      0.00      0.00      0.00 (VIOLATED: i
[c.valentin@ece05 test2]$
```

Figure 25. Where test2 stopped: the 6.2 violations, and fast hold under the hold floor.

First ask PrimeTime for hold fixes. `hold_both.tcl` asks for +0.15 ns of hold in both modes while keeping +0.5 ns of setup at the fast corner. `TRY_ONLY=1` stops the fix loop after PrimeTime, so the design does not change.

```
mkdir -p $RUN/hand && cd $RUN/hand
TRY_ONLY=1 MANUAL_CHANGES=~/flash/chip/margin/hold_both.tcl \
~/flash/flow/eco.sh $RUN fast manual
cp $RUN/signoff/eco_pt_fast_manual/eco_changes.tcl my_hold.tcl
```

```
c.valentin@ece05: ~/flash/runs/test2/hand

[c.valentin@ece05 hand]$ grep -E "^FLASH_(PT_ECO|ECO_TRY)" try_hold.log | cut -c1-104
FLASH_PT_ECO corner=fast type=manual mode=func setup_wns_before=11.610882 setup_wns_after=9.623190 hold_
FLASH_ECO_TRY_ONLY nothing applied; see /ecel/UFAD/c.valentin/flash/runs/test2/signoff/eco_pt_fast_manua
[c.valentin@ece05 hand]$ grep -c "insert_buffer\|^size_cell" my_hold.tcl
290
[c.valentin@ece05 hand]$
```

Figure 26. PrimeTime's proposal at the fast corner: 290 changes written, nothing applied.

Then test the list. `fc_pins.txt` names the memory outputs you will buffer in Fusion Compiler. PrimeTime cannot judge the load of a buffer it has not placed, so the test leaves those pins out.

```
printf "u_soc/u_imem/A_DOUT[15]\nu_soc/u_dmem/A_DOUT[25]\n" > fc_pins.txt
bash ~/flash/chip/margin/test_list.sh $RUN my_hold.tcl fc_pins.txt
```

```
c.valentin@ece05: ~/flash/runs/test2/hand
[c.valentin@ece05 hand]$ grep -E "^corner|^DROP|^MARGIN" test1.out
corner=fast mode=func bad=0 drc_viol=1 noise_viol=0 setup_wns=11.514 hold_wns=0.147
corner=slow mode=func bad=2 drc_viol=17 noise_viol=0 setup_wns=0.821 hold_wns=0.400
corner=typ mode=func bad=0 drc_viol=1 noise_viol=0 setup_wns=7.579 hold_wns=0.251
corner=fast mode=shift bad=0 drc_viol=1 noise_viol=0 setup_wns=11.514 hold_wns=0.145
corner=slow mode=shift bad=2 drc_viol=17 noise_viol=0 setup_wns=0.888 hold_wns=0.400
corner=typ mode=shift bad=0 drc_viol=1 noise_viol=0 setup_wns=7.629 hold_wns=0.248
DROP ctmTdsLR_1_33509 drc
DROP HFSBUF_83_29083 drc
MARGIN_GATE FAIL
[c.valentin@ece05 hand]$ grep "FLASH_DRC_DRIVER" test1.out | awk '{print $2, $5}' | head -8
u_soc/u_dmem/A_BM[10] sg13g2_nand2_1
u_soc/u_dmem/A_BM[11] sg13g2_nand2_1
u_soc/u_dmem/A_BM[12] sg13g2_nand2_1
u_soc/u_dmem/A_BM[13] sg13g2_nand2_1
u_soc/u_dmem/A_BM[14] sg13g2_nand2_1
u_soc/u_dmem/A_BM[15] sg13g2_nand2_1
u_soc/u_dmem/A_BM[8] sg13g2_nand2_1
u_soc/u_dmem/A_BM[9] sg13g2_nand2_1
[c.valentin@ece05 hand]$
```

Figure 27. The first test. Fast hold rises to +0.147 ns, but 2 changed cells break a transition limit at slow. The test names them and lists the weak drivers.

Fix one kind of problem at a time and test after each change. Drop the bad cells with `filter_changes.py`. Add one `size_cell` line per driver the test lists, one size larger. The block shows one. Put a delay cell in front of each endpoint under the hold floor.

```
python3 ~/flash/chip/margin/filter_changes.py my_hold.tcl my_hold2.tcl \
  $RUN/margin/test_my_hold/bad_cells_*.txt
cp my_hold2.tcl my_hold3.tcl
echo current_instance >> my_hold3.tcl
echo "size_cell {u_soc/u_core/u_ibex_core/A4285} {sg13g2_nand2_2}" >> my_hold3.tcl
echo "insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 \
  -new_net_names {net_HAND_H1} -new_cell_names {U_HAND_H1}" >> my_hold3.tcl
bash ~/flash/chip/margin/test_list.sh $RUN my_hold3.tcl fc_pins.txt
```

```
c.valentin@ece05: ~/flash/runs/test2/hand

[c.valentin@ece05 hand]$ tail -7 my_hold3.tcl | cut -c1-104
size_cell {u_soc/u_core/u_ibex_core/A4285} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4291} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4297} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4303} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4306} {sg13g2_nand2_2}
size_cell {u_soc/u_core/u_ibex_core/A4315} {sg13g2_nand2_2}
insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 -new_net_names {net_HAND_H1} -new_
[c.valentin@ece05 hand]$ grep -E "^corner|^FLASH_HOLD_END|^MARGIN" test3.out
corner=fast mode=func bad=0 drc_viol=1 noise_viol=0 setup_wns=11.514 hold_wns=0.068
corner=slow mode=func bad=0 drc_viol=2 noise_viol=0 setup_wns=0.821 hold_wns=0.220
corner=typ mode=func bad=0 drc_viol=1 noise_viol=0 setup_wns=7.579 hold_wns=0.132
corner=fast mode=shift bad=0 drc_viol=1 noise_viol=0 setup_wns=11.514 hold_wns=0.074
corner=slow mode=shift bad=0 drc_viol=2 noise_viol=0 setup_wns=0.888 hold_wns=0.233
corner=typ mode=shift bad=0 drc_viol=1 noise_viol=0 setup_wns=7.629 hold_wns=0.141
FLASH_HOLD_END u_soc/u_dmem/A_DIN[25] 0.069
FLASH_HOLD_END u_soc/u_dmem/A_DIN[25] 0.075
FLASH_HOLD_END u_soc/u_imem/A_DIN[0] 0.089
MARGIN_GATE FAIL
[c.valentin@ece05 hand]$
```

Figure 28. After dropping the bad cells, upsizing the listed drivers and adding one delay cell. The transition violations are gone, but one memory input fell under the hold floor.

Each fix moves something else. Here the larger drivers made the path to dmem/A_DIN[25] faster. One more delay cell there, and a second on imem/A_DIN[0], which was 0.001 ns short, bring every corner and mode through.

```
cp my_hold3.tcl my_hold4.tcl
echo "insert_buffer [get_pins {u_soc/u_dmem/A_DIN[25]}] sg13g2_dlygate4sd1_1 \
-new_net_names {net_HAND_H2} -new_cell_names {U_HAND_H2}" >> my_hold4.tcl
echo "insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 \
-new_net_names {net_HAND_H3} -new_cell_names {U_HAND_H3}" >> my_hold4.tcl
bash ~/flash/chip/margin/test_list.sh $RUN my_hold4.tcl fc_pins.txt
```

```
c.valentin@ece05: ~/flash/runs/test2/hand

[c.valentin@ece05 hand]$ tail -2 my_hold4.tcl | cut -c1-104
insert_buffer [get_pins {u_soc/u_dmem/A_DIN[25]}] sg13g2_dlygate4sd1_1 -new_net_names {net_HAND_H2} -new_
insert_buffer [get_pins {u_soc/u_imem/A_DIN[0]}] sg13g2_dlygate4sd1_1 -new_net_names {net_HAND_H3} -new_
[c.valentin@ece05 hand]$ grep -E "^corner|^FLASH_HOLD_END|^MARGIN" test4.out
corner=fast mode=func bad=0 drc_viol=1 noise_viol=0 setup_wns=11.514 hold_wns=0.092
corner=slow mode=func bad=0 drc_viol=2 noise_viol=0 setup_wns=0.821 hold_wns=0.251
corner=typ mode=func bad=0 drc_viol=1 noise_viol=0 setup_wns=7.579 hold_wns=0.152
corner=fast mode=shift bad=0 drc_viol=1 noise_viol=0 setup_wns=11.514 hold_wns=0.092
corner=slow mode=shift bad=0 drc_viol=2 noise_viol=0 setup_wns=0.888 hold_wns=0.251
corner=typ mode=shift bad=0 drc_viol=1 noise_viol=0 setup_wns=7.629 hold_wns=0.152
MARGIN_GATE PASS
[c.valentin@ece05 hand]$
```

Figure 29. The final test. Fast hold is +0.092 ns, slow setup keeps +0.821 ns, and the gate passes.

Last, apply it. The Fusion Compiler script is the tested list, a buffer on each memory output in fc_pins.txt, and sclk_swap.tcl, which swaps the flip-flop for one without the unused output. The fix loop applies it, then finishes and signs off the chip as in 7.1.


```
{ cat my_hold4.tcl; echo current_instance
echo "insert_buffer [get_pins {u_soc/u_imem/A_DOUT[15]]} sg13g2_buf_2 \
-new_net_names {net_HAND_D1} -new_cell_names {U_HAND_D1}"
echo "insert_buffer [get_pins {u_soc/u_dmem/A_DOUT[25]]} sg13g2_buf_2 \
-new_net_names {net_HAND_D2} -new_cell_names {U_HAND_D2}"
cat ~/flash/chip/margin/sclk_swap.tcl; } > my_fix.tcl
export AFTER_FC="METAL_FILL=mid bash ~/flash/flow/finish.sh . 75 bondpads"
FC_CHANGES=$PWD/my_fix.tcl ~/flash/flow/eco.sh $RUN slow fc
MARGIN=hand bash ~/flash/chip/make_chip.sh my_chip      # goes on to stage 8
```

```
c.valentin@ece05: ~/flash/runs/test2/hand
[c.valentin@ece05 hand]$ grep -E "^FLASH_ECO_(RELEASED|LEGALITY|APPLIED|KEPT)" apply.log
FLASH_ECO_RELEASED 65 of 65
FLASH_ECO_LEGALITY_PASS
FLASH_ECO_APPLIED 333
FLASH_ECO_KEPT 117 checks pass against 111 before
[c.valentin@ece05 hand]$ grep VIOLATED ../signoff/pt_slow/violators.rpt | cut -c1-80
u_soc/u_imem/A_DOUT[11]      0.06      0.06      0.00 (VIOLATED: i
[c.valentin@ece05 hand]$
```

Figure 30. The hand round's verdict on test2: kept, with 117 checks passing against 111. APPLIED counts the list plus the 9-line legality check the fix loop adds.

Fusion Compiler places and routes the changes for real, so the result can differ a little from the PrimeTime test. On test2 the kept round left the instruction memory output A_DOUT[11] at its capacitance limit, slack 0.00, in every run. make_chip.sh stopped at margin again and printed it. A one-line second list, tested and applied the same way, fixed it:

```
current_instance
insert_buffer [get_pins {u_soc/u_imem/A_DOUT[11]]} sg13g2_buf_2 \
-new_net_names {net_HAND_D3} -new_cell_names {U_HAND_D3}
```

```
c.valentin@ece05: ~/flash/runs/test2/hand
[c.valentin@ece05 hand]$ grep -E "^corner|^MARGIN" test5.out
corner=fast mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=11.502 hold_wns=0.094
corner=slow mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=0.797 hold_wns=0.255
corner=typ mode=func bad=0 drc_viol=0 noise_viol=0 setup_wns=7.563 hold_wns=0.154
corner=fast mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=11.502 hold_wns=0.094
corner=slow mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=0.852 hold_wns=0.255
corner=typ mode=shift bad=0 drc_viol=0 noise_viol=0 setup_wns=7.609 hold_wns=0.154
MARGIN_GATE PASS
[c.valentin@ece05 hand]$
```

Figure 31. The second list tested: 0 electrical violations and the gate passes.

Before you apply a later PrimeTime list, rename its new cells. Every PrimeTime session names them U_PTECO_1, U_PTECO_2 and so on, so two lists collide. margin_fix.sh shows the sed line that renames them.

Stage 8: Proving it works

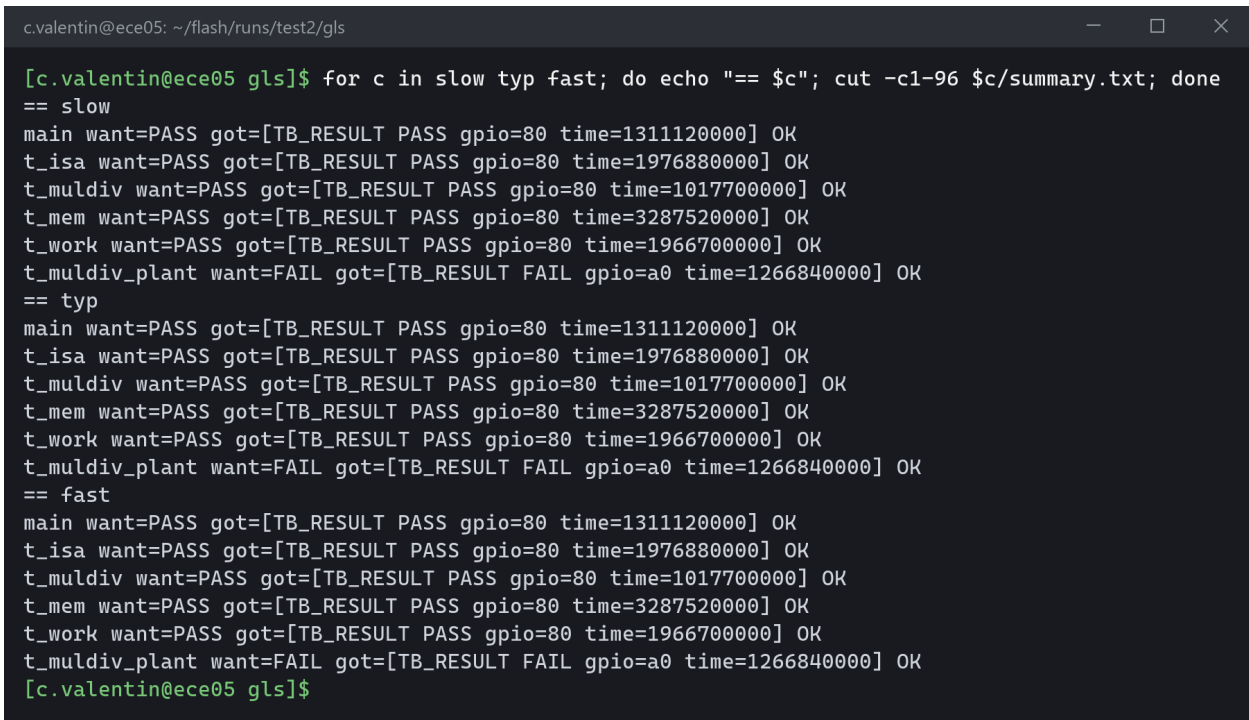
```
nohup bash ~/flash/chip/make_chip.sh my_chip > /dev/null 2>&1 &
```

This runs the margin rounds of stage 7 if signoff needs them, then the four proofs below, then writes the review drafts of stage 9 and stops. Each proof is also one script you can run again by itself.

8.1 Programs on the gates

```
bash ~/flash/chip/gls.sh $RUN
```

VCS runs six test programs on the final netlist with PrimeTime's delays at slow, typ and fast. `t_muldiv_plant` is built to fail, to prove a failure would show. It passed if it prints `GLS_PASS`.



```
c.valentin@ece05: ~/flash/runs/test2/gls
[c.valentin@ece05 gls]$ for c in slow typ fast; do echo "== $c"; cut -c1-96 $c/summary.txt; done
== slow
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
== typ
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
== fast
main want=PASS got=[TB_RESULT PASS gpio=80 time=1311120000] OK
t_isa want=PASS got=[TB_RESULT PASS gpio=80 time=1976880000] OK
t_muldiv want=PASS got=[TB_RESULT PASS gpio=80 time=1017700000] OK
t_mem want=PASS got=[TB_RESULT PASS gpio=80 time=3287520000] OK
t_work want=PASS got=[TB_RESULT PASS gpio=80 time=1966700000] OK
t_muldiv_plant want=FAIL got=[TB_RESULT FAIL gpio=a0 time=1266840000] OK
[c.valentin@ece05 gls]$
```

Figure 32. Every program at every corner, with the expected and actual result. The five real programs give 0x80 on the GPIO pins and `t_muldiv_plant` fails, as it must.

```

c.valentin@ece05: ~/flash/runs/test2/gls
[c.valentin@ece05 gls]$ grep -E "TB:|TB_RESULT" slow/t_work/sim.log | cut -c1-104
TB: loading 138 words from prog.hex
TB_RESULT PASS gpio=80 time=1966700000
[c.valentin@ece05 gls]$ grep -E "TB:|TB_RESULT" slow/t_muldiv_plant/sim.log | cut -c1-104
TB: loading 161 words from prog.hex
TB_RESULT FAIL gpio=a0 time=1266840000
[c.valentin@ece05 gls]$ grep -ih "timing violation" fast/*/sim.log | sed -E 's#.*(RM_[^"]+)", ([0-9]+).*#\1, li
ne \2#' | sort | uniq -c
105111 RM_IHPSG13_1P_1024x32_c2_bm_bist.v, line 164
108837 RM_IHPSG13_1P_1024x32_c2_bm_bist.v, line 165
[c.valentin@ece05 gls]$ tail -1 ../gls.log
GLS_PASS
[c.valentin@ece05 gls]$

```

Figure 33. One simulation. The testbench loads the program through the serial pins and reads the result on the GPIO pins. At fast, every timing message comes from two lines of IHP's memory model.

Those two lines use a fixed 1.0 ns placeholder for address setup and hold, while IHP's characterized library gives 0.32 to 0.34 ns hold, which PrimeTime meets.

8.2 Scan test

```
bash ~/flash/chip/atpg.sh $RUN
```

TestMAX makes stuck-at patterns for the scan chain. VCS replays every pattern twice, loaded straight into the flip-flops and shifted serially through the scan pins as a tester would. Both replays must show 0 mismatches.

```

c.valentin@ece05: ~/flash/runs/test2/signoff/atpg
[c.valentin@ece05 atpg]$ grep -E "^ (Detected|Possibly|Undetectable|ATPG untestable|Not detected|total faults|te
st coverage|#internal patterns) " summary.rpt
Detected                DT      111956
Possibly detected       PT         1
Undetectable            UD       639
ATPG untestable         AU      3890
Not detected            ND        16
total faults            116502
test coverage            96.63%
#internal patterns      634
[c.valentin@ece05 atpg]$ grep -h "XTB: Simulation" parallel.log fullserial.log | cut -c1-104
XTB: Simulation of 634 patterns completed with 0 mismatches (time: 190600.00 ns, cycles: 1906)
XTB: Simulation of 634 patterns completed with 0 mismatches (time: 118427600.00 ns, cycles: 1184276)
[c.valentin@ece05 atpg]$ tail -1 ../atpg.log
ATPG_DONE
[c.valentin@ece05 atpg]$

```

Figure 34. TestMAX's fault summary and both replays, each with 0 mismatches.

Test coverage leaves out the 639 faults that no input can make visible at an output. The patterns find wires stuck at 0 or 1. They do not reach inside the memories or test for faults that only make a path slow.

8.3 Clock crossings

```
bash ~/flash/chip/cdc.sh $RUN
```

SpyGlass CDC checks that every signal from the test board enters the clock domain through a synchronizer. It must find 0 unsynchronized crossings.

```
c.valentin@ece05: ~/flash/runs/test2/cdc
[c.valentin@ece05 cdc]$ grep -E "Ac_sync0[12]|Ar_sync01|Ar_syncdeassert01" verify.rpt | sed -E 's/^\[[0-9]+\] +(C[A-Za-z_0-9]+) +Info +[^\ ]+ +[0-9]+ +[0-9]+ +/\1: /; s/flash_soc\.//g; s/ +/ /g' | cut -c1-104 | head -5
Ac_sync02: Synchronized Crossing: destination flop sclk_s[0], clocked by clk, source primary input sclk,
Ac_sync02: Synchronized Crossing: destination flop sdi_s[0], clocked by clk, source primary input sdi, c
Ac_sync02: Synchronized Crossing: destination flop load_s[0], clocked by clk, source primary input load_
Ar_sync01: Reset signal 'rst_n' for 'clear' pin of flop 'sclk_s[0]', clocked by 'clk', is synchronized b
Ar_sync01: Reset signal 'rst_n' for 'clear' pin of flop 'u_core.core_busy_q[0]', clocked by 'clk', is sy
[c.valentin@ece05 cdc]$ grep -c "Ac_unsync" verify.rpt
0
[c.valentin@ece05 cdc]$
```

Figure 35. Synchronizers on all three loader inputs and the reset, and 0 unsynchronized crossings.

The package datasheet gives the timing rule the test board must follow on the loader pins.

8.4 Voltage drop and electromigration

```
source /apps/settings
export ANSYSLMD_LICENSE_FILE=1055@${SNPSLMD_LICENSE_FILE#*@}
bash ~/flash/chip/ir/ir.sh $RUN
```

RedHawk-SC needs an Ansys license, which /apps/settings does not set. On the ECE servers it is port 1055 on the Synopsys license server, which the export line builds. Set it before make_chip.sh too, or the build skips this check and says so in its log.

RedHawk-SC checks the power grid in five cases:

- current enters only through the straps next to the supply pads
- current enters through every strap from the pad ring
- the first case again, with the switching activity of the t_work program
- a control with a weakened grid, which must look worse
- a control with limits cut 1000 times, which must fail

No real case may break an EM limit. The worst drop must stay inside the 10% allowance.

```
c.valentin@ece05: ~/flash/runs/test2
[c.valentin@ece05 test2]$ grep worst_cell ir.log | sed -E 's/worst_cell=([0-9.]+) [^\ ]+ /worst=\1 V/; s/em_wire_max=/EM wire %=/; s/em_via_max=/via %=/; s/em_fail=/fails=/'
pads worst=1.173 V EM wire %=62.11 via %=35.63 fails=0
straps worst=1.19 V EM wire %=20.37 via %=12.86 fails=0
saif worst=1.187 V EM wire %=25.98 via %=17.68 fails=0
emcontrol worst=1.173 V EM wire %=6.211e+04 via %=3.563e+04 fails=5000
defect worst=1.167 V EM wire %=69.96 via %=34.71 fails=0
[c.valentin@ece05 test2]$
```

Figure 36. RedHawk-SC's result per case: the worst cell voltage on the 1.2 V supply, the worst wire and via current as a percent of IHP's limit, and the count of failing rows.

Case	Worst cell	Drop	Worst EM
pads only, 10% activity	1.173 V	27 mV, 2.3%	62.1% of the limit
every strap, 10% activity	1.190 V	10 mV	20.4%
pads only, t_work activity	1.187 V	13 mV, 1.1%	26.0%
control, weakened grid	1.167 V	33 mV	must be worse, and is
control, limits cut 1000 times			all 5,000 listed rows fail, as they must

No real case breaks an EM limit, and the worst drop is far inside the allowance. The model leaves out the pad ring's own rails and the package, and does not check fast-changing current.

Stage 9: Review records and final signoff

```
cat $RUN/review/SUMMARY.txt $RUN/review/EVIDENCE.txt
python3 ~/flash/chip/review/sign.py $RUN "Your Name"
bash ~/flash/chip/final_signoff.sh $RUN
```

Some things a tool can report but not judge. A person reads the evidence and signs a record for each. `make_chip.sh` leaves unsigned drafts in `$RUN/review` with the evidence beside them. Sign only if the evidence agrees with the basis `sign.py` lists. If `SUMMARY.txt` names an UNEXPECTED pin, do not sign until you know why that pin has no clock.

Record	What the person checks
timing_waivers.json	the two unlocked pins are the memories' unused self-test clocks
timing_coverage.json	every timing check PrimeTime skips has a reason
formality_coverage.json	the one point left out is the scan output, which the scan test covers
atpg/evidence.json	the coverage, the replays and every TestMAX warning

```
c.valentin@ece05: ~/flash/runs/test2
[c.valentin@ece05 test2]$ ls review
EVIDENCE.txt
formality_coverage.json
SUMMARY.txt
timing_coverage.json
timing_waivers.json
[c.valentin@ece05 test2]$ grep -E "unlocked pins|formality" review/SUMMARY.txt | cut -c1-104
unlocked pins drafted: u_soc/u_dmem/A_BIST_CLK u_soc/u_imem/A_BIST_CLK
formality: passing 2188, excluded 1 ['i:/WORK/flash_chip/scan_out', 'r:/WORK/flash_chip/scan_out'], cloc
formality unmatched: none reported
[c.valentin@ece05 test2]$ grep -h '"reviewed_by"' signoff/timing_waivers.json signoff/timing_coverage.json signoff/formality_coverage.json signoff/atpg/evidence.json
"reviewed_by": "Cesar Valentin",
"reviewed_by": "Cesar Valentin",
"reviewed_by": "Cesar Valentin",
"reviewed_by": "Cesar Valentin",
[c.valentin@ece05 test2]$
```

Figure 37. The drafts, the summary naming the two unlocked pins and the one Formality exclusion, and the four records after signing.

The signed records feed the checker, so `final_signoff.sh` reruns every signoff step. Stage 9 is done when the last line of `signoff/CHECK.txt` says `SIGNOFF CLEAN`.

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ for c in slow typ fast slow_shift typ_shift fast_shift; do printf "%-11s" $c; grep -
h "^FLASH_PT corner" pt_${c}/pt.log | head -1 | grep -o "setup_wns=[-0-9.]* hold_wns=[-0-9.]*"; done
slow      setup_wns=0.797503 hold_wns=0.255708
typ       setup_wns=7.563652 hold_wns=0.154871
fast      setup_wns=11.502912 hold_wns=0.094022
slow_shift setup_wns=0.852478 hold_wns=0.255708
typ_shift setup_wns=7.609718 hold_wns=0.154871
fast_shift setup_wns=11.502912 hold_wns=0.094022
[c.valentin@ece05 signoff]$ for c in slow typ fast; do grep -h "^FLASH_PT_NOISE" pt_${c}/pt.log; done
FLASH_PT_NOISE corner=slow worst_slack=0.03 violators=0
FLASH_PT_NOISE corner=typ worst_slack=0.04 violators=0
FLASH_PT_NOISE corner=fast worst_slack=0.03 violators=0
[c.valentin@ece05 signoff]$ grep -c VIOLATED pt_slow/violators.rpt pt_typ/violators.rpt pt_fast/violators.rpt
pt_slow/violators.rpt:0
pt_typ/violators.rpt:0
pt_fast/violators.rpt:0
[c.valentin@ece05 signoff]$
```

Figure 38. PrimeTime after the final signoff: 0 noise violators and 0 electrical violations in every run.

```
c.valentin@ece05: ~/flash/runs/test2/signoff

[c.valentin@ece05 signoff]$ grep -E "^(fc_flow|chip_drc|chip_precheck|chip_lvs_completion|formality|atpg_coverag
e|formality_coverage|electrical_slow|pt_slow|noise_slow|timing_waivers|pt_report_analysis_coverage_slow) " CHECK
.txt | cut -c1-104
fc_flow          PASS      done=True errors=0
pt_slow          PASS      period=20.0ns setup_wns=0.798 hold_wns=0.256 spef_nets=
noise_slow       PASS      worst_slack=0.03 violators=0
formality        SUCCEEDED Passing=2188
chip_drc          PASS      klayout violations=0 recommended_only=0
chip_precheck     PASS      klayout violations=0 recommended_only=0
electrical_slow   PASS      unwaived=0 waived=0
pt_report_analysis_coverage_slow PASS  reviewed reasons=conditional_arc:304,constant_at_pin:6,
formality_coverage PASS      passing=2188 excluded=1 clock_gate_latches=0; requires
atpg_coverage     PASS      patterns=634 test_coverage=96.63 total_faults=116502 St
chip_lvs_completion PASS      latest match verdict and successful exit required
[c.valentin@ece05 signoff]$ grep -c " PASS " CHECK.txt
139
[c.valentin@ece05 signoff]$ tail -1 CHECK.txt
SIGNOFF CLEAN: 139 of 139 checks pass
[c.valentin@ece05 signoff]$
```

Figure 39. The checker after the final signoff, ending in `SIGNOFF CLEAN`.

Stage 10: The release package

```
bash ~/flash/chip/package/package.sh $RUN
```

It first checks that every file signoff judged is unchanged since, by the checksums in `CHECK.txt`. Then it fills IHP's release folder, `$RUN/package/IHP__RVSoC8787`. It saves the GDS with IHP's settings, renames the top cell to the IP name, compares that GDS shape for shape and label for label with the signed-off chip, and runs IHP's pre-check on it. It passed if it ends with `PACKAGE_OK`. The package's `doc/Datasheet.md` gives the bring-up steps.

```

c.valentin@ece05: ~/flash/runs/test2/package
[c.valentin@ece05 package]$ ls IHP__RVSoC8787 IHP__RVSoC8787/release/v.1.0.0/gds IHP__RVSoC8787/release/v.1.0.0/
netlist
IHP__RVSoC8787:
doc
README.md
release
RVSoC8787-main

IHP__RVSoC8787/release/v.1.0.0/gds:
RVSoC8787.gds

IHP__RVSoC8787/release/v.1.0.0/netlist:
flash_chip.v
RVSoC8787.cdl
[c.valentin@ece05 package]$ grep -h "^LAYERS\|^TEXTS" xor.txt texts.txt
LAYERS 59 DIFFERING 8
TEXTS 1676862 1676862 ONLY_A 0 ONLY_B 0
[c.valentin@ece05 package]$ tail -3 ../package.log
PRECHECK_EXIT=0
PRECHECK_MARKERS=0
PACKAGE_OK /eceL/UFAD/c.valentin/flash/runs/test2/package/IHP__RVSoC8787
[c.valentin@ece05 package]$

```

Figure 40. The release folder, the comparison with the signed-off chip and the package log. Every label matches, the pre-check finds no markers, and the script ends with PACKAGE_OK.

The shape log lists eight layers as differing. Each holds only labels, with 0 area and 0 polygons of difference.

Results of the fresh build

These are the results of test2, the build fixed by hand in Stage 7.2. test1, fixed by the automatic rounds, also ended SIGNOFF CLEAN, with slow setup +0.707 ns and fast hold +0.116 ns.

Check	Result
final signoff	SIGNOFF CLEAN, 139 of 139 checks
slow setup slack	+0.797 ns at the 20 ns period
hold slack, slow, typ and fast	+0.256, +0.155 and +0.094 ns, worst of both modes
noise and electrical limits	0 violations at every corner
Formality	2,188 compare points pass
chip DRC and IHP pre-check	0 and 0
block and whole-chip LVS	match
metal density	44.6% to 55.0% on every metal layer
power	3.45 mW at the typical corner
programs on the gates	all pass at slow, typ and fast, and the control fails
scan test	634 patterns, 96.63% test coverage, 0 mismatches
clock crossings	0 unsynchronized
worst static supply drop	27 mV, 2.3%

Check	Result
worst EM	62.1% of IHP's limit
die size	1708 by 1753 μm

This signoff leaves three things unproven. IHP publishes no factory-qualified worst and best wire models. LVS does not check inside IHP's memory and pad cells. The voltage-drop check is static only.

Labs: change one setting

Once your first chip is through place and route, change one setting at a time and see what it does. Each lab makes a new run, adds one line to its `run_cfg.tcl`, and stops after place and route, about 15 minutes. The first lab:

```
bash ~/flash/chip/new_run.sh lab_layers
echo "set MAX_LAYER Metal4" >> ~/flash/runs/lab_layers/run_cfg.tcl
STOP_AT=route bash ~/flash/chip/make_chip.sh lab_layers
```

Then compare it with your first run. These three lines give the routing errors, the total wire length and Fusion Compiler's worst setup slack for every run:

```
cd ~/flash/runs
grep -H "Total number of DRCs =" */rpt/check_routes.rpt
grep -H "Total Wire Length" */rpt/check_routes.rpt
grep -H "Design *(Setup)" */rpt/final_qor.rpt
```

Lab	Line added	Route DRCs	Wire length	Setup slack at the target
my_chip, the base	none	0	756,681 μm	-0.107 ns at 18 ns
lab_layers	set MAX_LAYER Metal4	0	790,741 μm	-0.047 ns at 18 ns
lab_clock	set PNR_MARGIN 0.25	0	760,143 μm	-0.031 ns at 15 ns
lab_ndr	set RUN_NDR {spacing 2 width 1}	0	1,189,896 μm	-0.092 ns at 18 ns

Three routing layers instead of four: the router put no wire on Metal5 and still routed with 0 errors, at 4.5% more wire. A harder target, 15 ns instead of 18 ns: Fusion Compiler came within 0.03 ns of it with almost the same cells and wire, so this design has timing to spare. Double spacing on every signal route: still 0 errors, but 57% more wire, and fast hold slipped just under 0 at three endpoints. Wider spacing is a tool for the few nets that need it, not for all of them. Setup slack moves by a few hundredths of a nanosecond from run to run, so do not read a small change as a result.

Command summary

```
git clone https://github.com/XxMasterepicxX/ibex-sg13g2-chip.git ~/flash
bash ~/flash/setup/setup.sh                                once, about two hours
source /apps/settings
export ANSYSLMD_LICENSE_FILE=1055@${SNPSLMD_LICENSE_FILE#*@}  for stage 8.4
nohup bash ~/flash/chip/make_chip.sh my_chip &              makes the run, then stages 3 to 8
RUN=~/flash/runs/my_chip
python3 ~/flash/chip/review/sign.py $RUN "Your Name"        after reading the drafts
bash ~/flash/chip/final_signoff.sh $RUN                      ends with SIGNOFF CLEAN
bash ~/flash/chip/package/package.sh $RUN                   ends with PACKAGE_OK
```